

Relazione - Versione 7: LLM Router

Introduzione

La Versione 7 introduce l'LLM Router.

Nella versione precedente, il progetto ha introdotto il Tool Registry. L'agente disponeva già di una fonte centrale per funzioni, parametri, descrizioni e sinonimi degli strumenti, mentre il Router selezionava ancora gli strumenti tramite regole definite manualmente.

La Versione 7 mantiene la stessa architettura, ma cambia il modo in cui il Router comprende la richiesta dell'utente. Invece di assegnare punteggi a sinonimi definiti manualmente, il Router utilizza ora un piccolo modello linguistico preaddestrato per selezionare lo strumento registrato più appropriato.

Il modello linguistico viene eseguito localmente nel notebook. Viene scaricato da Hugging Face e non utilizza un servizio esterno di inferenza tramite API.

Obiettivo della Versione 7

L'obiettivo principale della Versione 7 è introdurre il routing degli strumenti basato su un modello linguistico mantenendo invariato il resto dell'agente.

Il Router deve utilizzare i nomi e le descrizioni memorizzati nel Tool Registry per capire quale strumento può gestire la richiesta. Se nessuno strumento registrato è appropriato, deve restituire nessuno strumento.

Solo il Router utilizza un LLM in questa versione. Il Planner rimane basato su regole, il Generic Parser rimane basato su regole e l'esecuzione e la gestione della memoria continuano a funzionare come prima.

Cosa cambia rispetto alla Versione 6

La Versione 6 ha centralizzato la gestione degli strumenti tramite il Tool Registry. Il Router dipendeva ancora da sinonimi registrati, punteggi e priorità per selezionare uno strumento.

La Versione 7 rimuove dal Router questa logica di selezione basata su regole. Il Tool Registry rimane invariato, ma il Router legge ora i nomi e le descrizioni degli strumenti e li presenta a un modello linguistico locale.

I sinonimi restano memorizzati nel registro per continuità con la Versione 6, ma il Router della Versione 7 non li utilizza più per prendere la decisione di routing.

Nuovo concetto: LLM Router

L'LLM Router trasforma la selezione dello strumento in un piccolo compito di comprensione del linguaggio. La richiesta dell'utente e le descrizioni degli strumenti disponibili vengono inserite in un prompt di routing e al modello viene chiesto di restituire un solo nome consentito oppure none.

La Versione 7 utilizza il seguente modello locale:

```
MODEL_NAME = "Qwen/Qwen2.5-0.5B-Instruct"
```

Modello e tokenizer vengono caricati con Transformers. Il runtime utilizza CUDA quando disponibile e altrimenti passa alla CPU:

```
device = "cuda" if torch.cuda.is_available() else "cpu"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME).to(device)
model.eval()
```

Dopo il caricamento, il modello viene utilizzato localmente. Hugging Face serve per ottenere i file del modello preaddestrato, non come servizio esterno di inferenza.

Come l'LLM Router utilizza il Tool Registry

Il Tool Registry rimane la fonte condivisa delle informazioni sugli strumenti. Per il routing, la Versione 7 costruisce un elenco compatto che contiene il nome e la descrizione di ogni strumento registrato.

```
for tool_name, tool_info in tool_registry.items():
    available_tools.append(
        f"- {tool_name}: {tool_info['description']}"
    )
```

Il prompt del Router elenca anche le risposte consentite e indica esplicitamente al modello di restituire none quando nessuno strumento può gestire direttamente la richiesta.

Il modello genera una risposta breve e deterministica con il campionamento disattivato. Il Router convalida poi il testo generato confrontandolo con i nomi degli strumenti registrati.

```
outputs = model.generate(
    **inputs,
    max_new_tokens=8,
    do_sample=False
)

for tool_name in tool_registry:
    if re.search(rf"\b{re.escape(tool_name)}\b", response):
        return tool_name

return None
```

Questo controllo finale non decide l'intento. La decisione semantica viene presa dal modello linguistico; il codice Python converte soltanto la risposta generata in un nome di strumento valido.

Architettura completa nella Versione 7

Il flusso di esecuzione diventa:

```
Richiesta -> LLM Router -> Planner -> Generic Parser
              -> Executor -> Memory Manager -> Memoria
                      |
                      v
                      Output
```

Responsabilità dei componenti

Le responsabilità nella Versione 7 sono:

Tool Registry = memorizza gli strumenti e i relativi metadati LLM Router = seleziona semanticamente lo strumento usando il modello linguistico locale Planner = crea il piano di esecuzione usando i metadati registrati Generic Parser = estrae gli argomenti richiesti dallo strumento selezionato Executor = convalida ed esegue la funzione registrata Memory Manager = salva, legge e pulisce la memoria Agent = coordina Memory = conserva lo stato

L'Agent rimane il coordinatore. La nuova intelligenza viene volutamente limitata al passaggio di routing, così il cambiamento architetturale resta semplice da comprendere.

Stato dell'Agent nella Versione 7

Lo stato dell'agente mantiene la stessa struttura utilizzata nelle versioni precedenti. L'introduzione dell'LLM Router non richiede una nuova chiave nello stato.

```
{
  "request": "Could you calculate the product of 2 and 3?",
  "plan": {
    "tool": "multiplication",
    "reason": "Multiply two numbers.",
    "needs_arguments": ["a", "b"]
  },
  "action": "multiplication",
  "arguments": {"a": 2, "b": 3},
  "result": 6
}
```

La differenza è nel modo in cui viene selezionata l'action: il Router interpreta ora la richiesta con il modello locale prima che il Planner crei il piano.

Richieste non supportate

Le richieste non supportate vengono ora valutate semanticamente invece che tramite corrispondenza con i sinonimi.

Per esempio:

```
agent("What is the weather in Rome?")
```

Nel registro non esiste alcuno strumento per il meteo. L'LLM Router quindi non restituisce alcuno strumento, il Planner crea un piano senza un'azione selezionata e l'Agent restituisce il risultato di fallback già esistente.

L'interazione viene comunque salvata tramite il Memory Manager esattamente come nella Versione 6.

Test

I test verificano sia la compatibilità con l'architettura precedente sia il nuovo comportamento di routing semantico:

```
clear_memory()
tool_registry

agent("What is 2 + 3?")
agent("Hello, what is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
agent("Could you calculate the product of 2 and 3?")
```

```
get_memory()  
get_last_interaction()
```

Questi test verificano che:

- l'LLM Router selezioni addition per una richiesta aritmetica diretta;
- l'LLM Router identifichi addition come intento principale anche quando la richiesta inizia con un saluto;
- lo strumento greeting continui a funzionare con l'estrazione del nome basata su regole;
- una richiesta meteo non supportata non selezioni alcuno strumento;
- la parola product venga compresa come multiplication anche se non è uno dei sinonimi di routing della multiplication nella Versione 6;
- Planner, Generic Parser, Executor, Memory Manager e Agent continuino a funzionare senza cambiamenti architetturali;
- la memoria continui a conservare lo stato completo di ogni interazione.

Cosa insegna la Versione 7

La Versione 7 insegna il primo utilizzo pratico di un modello linguistico all'interno dell'architettura dell'agente:

il routing semantico può sostituire regole di intento mantenute manualmente

Il Router non necessita più di un punteggio basato sui sinonimi per ogni possibile modo in cui l'utente può esprimere lo stesso intento. Le descrizioni degli strumenti diventano contesto leggibile dalla macchina per selezionare un'azione.

Allo stesso tempo, la responsabilità dell'LLM resta limitata. Il modello seleziona lo strumento, ma non estrae ancora gli argomenti, non crea piani multi-step, non esegue funzioni e non gestisce la memoria.

Limitazioni

La Versione 7 introduce un LLM, ma rimangono alcune limitazioni:

- il piccolo modello locale può ancora commettere errori di routing;
- il Planner rimane basato su regole;
- il Generic Parser usa ancora estrazione manuale;
- il Tool Registry resta in memoria e la registrazione è manuale;
- la memoria rimane una semplice lista. l'agente continua a eseguire una sola azione alla volta.

Conclusione

La Versione 7 è un'evoluzione naturale della Versione 6.

Il Tool Registry rimane la fonte centrale delle informazioni sugli strumenti, ma il routing non è più basato su punteggi di sinonimi definiti manualmente. Un modello linguistico preaddestrato eseguito localmente interpreta ora la richiesta dell'utente e seleziona lo strumento registrato in grado di gestirla.

Il resto dell'architettura rimane volutamente invariato: il Planner è ancora basato su regole, il Generic Parser continua a estrarre gli argomenti con regole esplicite,

l'Executor esegue la funzione registrata e il Memory Manager memorizza lo stato delle interazioni.

Il test con product dimostra il principale vantaggio del nuovo Router: può riconoscere semanticamente un intento che non era codificato esplicitamente come sinonimo di routing. Il test sul meteo mostra inoltre che il Router può restituire nessuno strumento quando il registro non contiene una capacità appropriata.

Il prossimo passo sarà la Versione 8, nella quale il progetto introdurrà un LLM Parser e sposterà l'estrazione degli argomenti dalle regole manuali al parsing basato su un modello linguistico.